



Plain Text Files

Part 3: Working With JSON Files

Some files use objects and lists to store data for hierarchal access.

JSON Files

A **JSON**, or **JavaScript Object Notation**, file stores structured data using a combination of **objects**, **arrays**, and **values**.

JSON is commonly used when data has relationships or nested structure that would be awkward to store in plain text or CSV files.

It is often used by websites, APIs, configuration files, and programs that need to save complex data.

Objects in JSON

A JSON **object** stores related data using **key-value pairs**.

Each key acts like a label for a value.

In Python, JSON objects are loaded as dictionaries.

```
{  
    "name": "Alex",  
    "score": 82  
}
```

An example of a JSON Object
(this is not Python code)

Arrays in JSON

A JSON **array** stores multiple values in order.

Arrays can contain simple values, objects, or even other arrays.

In Python, JSON arrays are loaded as lists.

```
[  
    "Alex",  
    "Sam",  
    "Jordan"  
]
```

An example of a JSON Array
(this is not Python code)

Values in JSON

A JSON **value** is an individual piece of data stored inside an object or array.

Values can include strings, numbers, booleans, **null**, objects, or arrays.

In Python, these are converted into their equivalent Python datatypes.

"Alex"

82

true

Examples of JSON Values.
They are not formatted properly, this is just to show what kinds of values you might have
(this is not Python code)

```
{  
  "class_name": "Computer Science 15",  
  "teacher": "Mr. Forsyth",  
  "students": [  
    {  
      "name": "Alex",  
      "score": 82,  
      "passed": true  
    },  
    {  
      "name": "Sam",  
      "score": 91,  
      "passed": true  
    },  
    {  
      "name": "Jordan",  
      "score": 48,  
      "passed": false  
    }  
  ]  
}
```

Nesting in JSON Files

JSON files can contain objects inside arrays, arrays inside objects, and multiple levels of nested data.

This makes JSON very flexible, but it also means you must be precise about where a value is located.

Compared with plain text or CSV files, JSON structures can be more difficult to navigate because accessing data may require several keys and indices.

Nesting in JSON Files *continued...*

The

main object

contains the values:

class_name

teacher

students

```
{  
  "class_name": "Computer Science 15",  
  "teacher": "Mr. Forsyth",  
  "students": [  
    {  
      "name": "Alex",  
      "score": 82,  
      "passed": true  
    },  
    {  
      "name": "Sam",  
      "score": 91,  
      "passed": true  
    }  
  ]  
}
```

Nesting in JSON Files *continued...*

The

students value

contains a list of

student objects

```
{  
  "class_name": "Computer Science 15",  
  "teacher": "Mr. Forsyth",  
  "students": [  
    {  
      "name": "Alex",  
      "score": 82,  
      "passed": true  
    },  
    {  
      "name": "Sam",  
      "score": 91,  
      "passed": true  
    }  
  ]  
}
```

Nesting in JSON Files *continued...*

Each

student object

contains the values

name

score

passed

```
{  
  "class_name": "Computer Science 15",  
  "teacher": "Mr. Forsyth",  
  "students": [  
    {  
      "name": "Alex",  
      "score": 82,  
      "passed": true  
    },  
    {  
      "name": "Sam",  
      "score": 91,  
      "passed": true  
    }  
  ]  
}
```

Nesting in JSON Files *continued...*

The **big take away** is that a **JSON file** will contain a **main object** that has other **values**, **arrays**, and even **other objects nested inside of it!**

Importing **json** into Python

Python includes a built-in **json** module for reading and writing JSON files.

It must be imported before using JSON-specific tools such as **load()** and **dump()**.

```
import json
```

Reading a JSON File in "r" Mode

A JSON file can be opened in read mode using "r".

The **with** statement automatically closes the file when the block finishes.

At this point, the file is open but the JSON data has not yet been converted into Python objects.

```
import json

with open("students.json", "r") as file:
    pass
```

Loading the File as a Variable

The `json.load()` function reads the JSON file and reconstructs its structure using Python objects.

JSON objects become dictionaries, arrays become lists, and values become normal Python values.

The complete data structure can then be stored in a variable.

```
import json

with open("students.json", "r") as file:
    data = json.load(file)
```

Viewing the Data Structure

```
# main.py

import json

with open("students.json", "r") as file:
    data = json.load(file)

print(data)
```

Output

(When it isn't formatted it prints as one line)

```
{'class_name': 'Computer
Science 15', 'teacher':
'Mr. Forsyth', 'students':
[{'name': 'Alex', 'score':
82, 'passed': True},
{'name': 'Sam', 'score':
91, 'passed': True},
{'name': 'Jordan', 'score':
48, 'passed': False}]}
```

Viewing the Data Structure

```
# main.py

import json

with open("students.json", "r") as file:
    data = json.load(file)

print(data)
```

Output

(formatted by Mr. Forsyth - normally it prints as one line)

```
{
  "class_name": "Computer Science 15",
  "teacher": "Mr. Forsyth",
  "students": [
    {
      "name": "Alex",
      "score": 82,
      "passed": true
    },
    {
      "name": "Sam",
      "score": 91,
      "passed": true
    },
    {
      "name": "Jordan",
      "score": 48,
      "passed": false
    }
  ]
}
```

Processing Objects from a JSON File

JSON objects become Python dictionaries.

Their values can be accessed using dictionary keys.

If an object is nested, you must first access the part of the structure that contains it.

```
import json

with open("students.json", "r") as file:
    data = json.load(file)

print(data["class_name"])
print(data["teacher"])
```

OUTPUT

```
Computer Science 15
Mr. Forsyth
```

Processing Objects from a JSON File

students.json

```
{
  "class_name": "Computer Science 15",
  "teacher": "Mr. Forsyth",
  "students": [
    {
      "name": "Alex",
      "score": 82,
      "passed": true
    },
    {
      "name": "Sam",
      "score": 91,
      "passed": true
    },
    {
      "name": "Jordan",
      "score": 48,
      "passed": false
    }
  ]
}
```

main.py

```
import json
```

```
with open("students.json", "r") as
file:
```

```
    data = json.load(file)
```

```
print(data["class name"])
```

```
print(data["teacher"])
```

OUTPUT

Computer Science 15

Mr. Forsyth

Processing Arrays from a JSON File

JSON arrays become Python lists.

They can be traversed with loops or accessed using list indices.

If the array contains objects, each item in the list will be a dictionary.

```
import json

with open("students.json", "r") as file:
    data = json.load(file)

for student in data["students"]:
    print(student["name"])
```

OUTPUT

```
Alex
Sam
Jordan
```

Processing Arrays from a JSON File

students.json

```
{
  "class_name": "Computer Science 15",
  "teacher": "Mr. Forsyth",
  "students": [
    {
      "name": "Alex",
      "score": 82,
      "passed": true
    },
    {
      "name": "Sam",
      "score": 91,
      "passed": true
    },
    {
      "name": "Jordan",
      "score": 48,
      "passed": false
    }
  ]
}
```

main.py

```
import json
```

```
with open("students.json", "r") as file:
    data = json.load(file)
```

```
for student in data["students"]:
    print(student["name"])
```

OUTPUT

```
Alex
Sam
Jordan
```

Processing Values from a JSON File

Values inside a JSON structure can be accessed by combining dictionary keys and list indices.

The exact path depends on how deeply the value is nested.

This is why understanding the structure of the JSON file is important.

```
import json

with open("students.json", "r") as file:
    data = json.load(file)

print(data["students"][1]["score"])
```

OUTPUT

91

Processing Values from a JSON File

students.json

```
{
  "class_name": "Computer Science 15",
  "teacher": "Mr. Forsyth",
  "students": [
    {
      "name": "Alex",
      "score": 82,
      "passed": true
    },
    {
      "name": "Sam",
      "score": 91,
      "passed": true
    },
    {
      "name": "Jordan",
      "score": 48,
      "passed": false
    }
  ]
}
```

A diagram illustrating the JSON structure. A large blue bracket on the right groups the entire JSON object. A yellow bracket on the right groups the 'students' array. A green bracket on the right groups the individual student objects within the array. A red arrow points from the 'score' field of the second student object ('Sam') to the value 91.

main.py

```
import json
```

```
with open("students.json", "r") as file:
    data = json.load(file)
```

```
print(data["students">[[1]["score"]])
```

OUTPUT

91

Writing JSON Files

Writing a JSON file usually involves three steps.

1. Create the data structure in Python.

2. Open a file in `"w"` mode.

3. Use `json.dump()` to convert the Python structure into JSON and write it to the file.

1

- Create a data structure to contain the data you want to convert to a JSON

2

- Open a JSON file in `"w"` mode.

3

- Dump the data structure as a JSON file.

1. Create a Data Structure

Before writing JSON, the structure must be created carefully using Python dictionaries, lists, and values.

The way these objects are nested determines how the final JSON file will be organized.

Being precise at this stage is important because a poorly planned structure can make the data difficult to access later.

```
data = {  
    "name": "Mr. Forsyth",  
    "experience": 150,  
    "inventory": [  
        "sword",  
        "armor",  
        "shield"  
    ]  
}
```

2. Writing the File in "w" Mode

A JSON file can be opened in write mode using "w".

If the file does not exist, Python creates it.

If it already exists, its current contents are replaced.

```
import json

with open("player.json", "w") as file:
    pass
```

3. Dumping the File

The `json.dump()` function converts a Python data structure into JSON and writes it into an open file.

The `indent=4` argument adds indentation to make the file easier for people to read.

Without indentation, the same data may be written on a single long line.

```
import json

data = {
    "name": "Mr. Forsyth",
    "experience": 150,
    "inventory": [
        "sword",
        "armor",
        "shield"
    ]
}

with open("player.json", "w") as file:
    json.dump(data, file, indent=4)
```

3. Dumping the File

```
# main.py

import json

data = {
    "name": "Mr. Forsyth",
    "experience": 150,
    "inventory": [
        "sword",
        "armor",
        "shield"
    ]
}

with open("player.json", "w") as file:
    json.dump(data, file, indent=4)
```

```
player.json

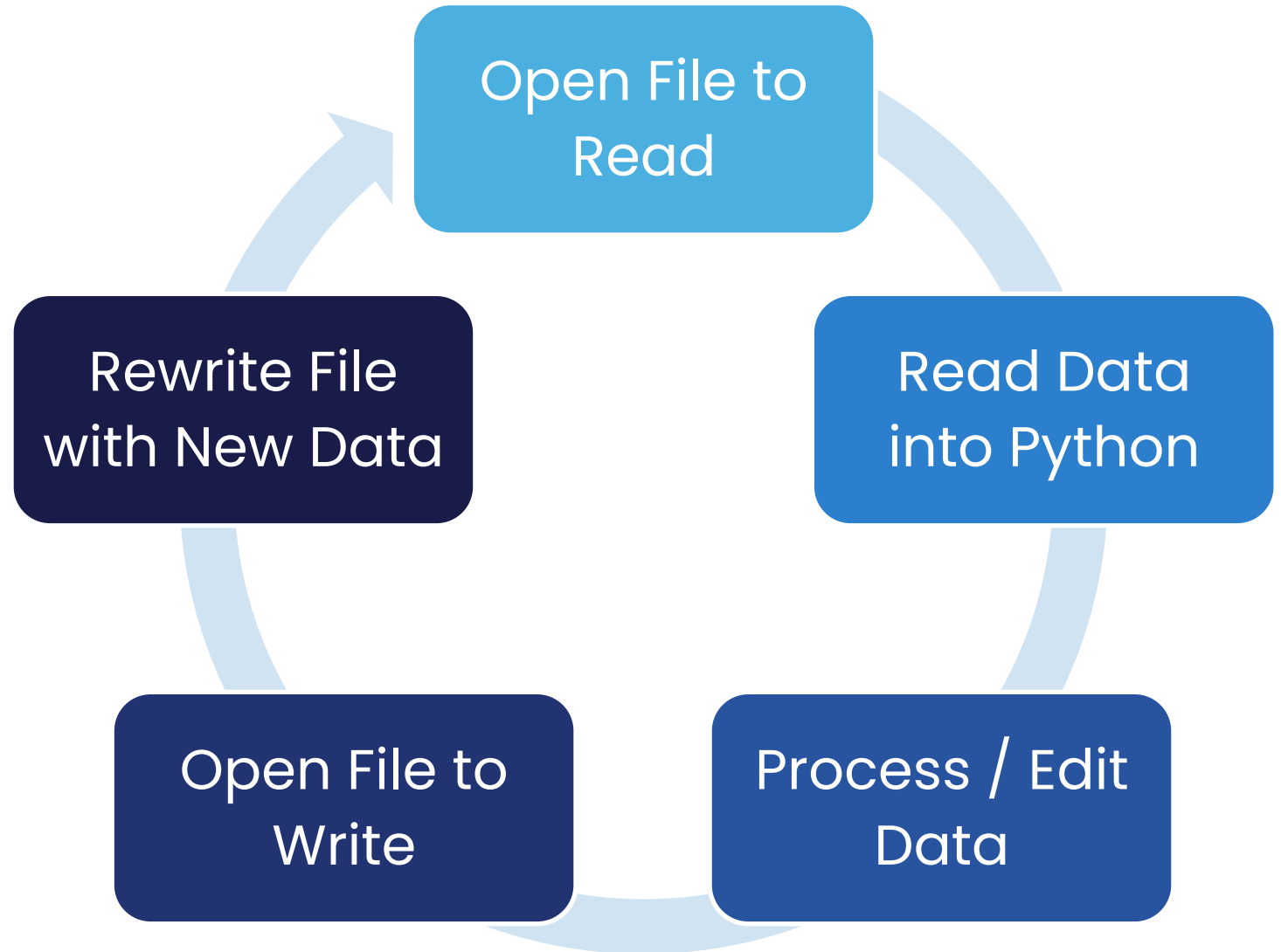
{
    "name": "Mr. Forsyth",
    "experience": 150,
    "inventory": [
        "sword",
        "armor",
        "shield"
    ]
}
```

Reading and Writing Files

A common file workflow is **read, process, write**.

The program first reads data from a file and stores it in Python.

It then processes or modifies that data before writing the updated information back to a file.



Example of Reading and Writing

```
import json

with open("students.json", "r") as file:
    data = json.load(file)

for student in data["students"]:
    student["score"] += 5

with open("students.json", "w") as file:
    json.dump(data, file, indent=4)
```

Wrap Up

JSON files store structured data using **objects, arrays, and values**.

In Python, these are commonly loaded as dictionaries, lists, and normal Python datatypes using `json.load()`.

Because JSON data can be deeply nested, accessing values requires carefully following the correct keys and indices.

To save JSON data, create the Python structure, open the file in `"w"` mode, and use `json.dump()` to write it.